Security Assessment & Formal Verification Report

Cozy

October / December 2025

Prepared for Cozy Finance

Table of Contents

Project Summary 5
Project Scope 5
Project Overview 5
Protocol Overview 8
Threat and Security Overview 9
1. Solvency & Balance-Sheet Consistency 9
2. Fair Value, Pro-Rata Redemptions & Loss Socialization 10
3. Authorization, Governance Binding & State-Machine Safety 10
4. Time Locks, Delayed Actions & Signature / Replay Safety 11
5. Specification Consistency: Previews, Execution & Edge-Case Equivalence 12
6. External Integration & Reentrancy-Style Surface Risks (Validated Entry Only) 13
Findings Summary 14
Severity Matrix 14
Detailed Findings 15
Medium Severity Issues 16
M-01 Rewards are not dripped upon stake token transfers 16
M-02 SafetyModule can add a controller that is not registered on the manager 17
Informational Issues 18
I-01. Module can be raised with non-reset expired triggers 18
I-02. Some config updates can be queued while module is triggered 19
I-03.A safetyModule can register an unlimited number of controllers in the manager 20
I-04. Controller registration can be front-runned and DOSed for a SafetyModule by another one 21
I-05. Rewards can round down to 0 for small amounts when adding pools in claimRewards 22
Formal Verification 25
Verification Methodology - Solidity 25
Verification Notations 26
General Assumptions and Simplifications 26
Formal Verification Properties Overview 28
Detailed Properties 35
SafetyModule 35
SM-01. Token balances are at least what has been accounted for 35
SM-02. Accounting for deposits works correctly 35
SM-03. State is triggered exactly when raises are pending 36
SM-04. Fees only drip when the SafetyModule is active 36
SM-05. Reserve pool accounting corresponds to actual balance 37
SM-06. Only drip methods drip 37
SM-07. Only authorized callers can unpause 38
SM-08. Account for redemption completion is correct 38
SM-09. Delegated redemption can not be replayed 39

SM-10. Only authorized users can skim....39
SM-11. Redemption fees are at most the redeemed amount....39
SM-12. Can not redeem when triggered....40
SM-13. Authorized caller can pause....40
SM-14. Claimed fees are transferred to the owner....41
SM-15. Redemptions hold the correct scaling factors....41
SM-16. Pending withdrawals are ignored when redeeming....42
SM-17. Assets amounts follow the SafetyModules balance....42
SM-18. Unpause leaves the pause state....43
SM-19. Only claim methods can claim fees....43
SM-20. Calling trigger enters the triggered state....44
SM-21. Redeeming transfers the expected amount of assets....44
SM-22. Only the owner can complete a redemption....44
SM-23. Only the manager can claim with a custom drip model....45
SM-24. Never call the SharedSafetyModule when it is zero....45
SM-25. An unauthorized caller can not pause....46
SM-26. Only complete redemptions after the delay....46
SM-27. Claimed fees are at most the accumulated fees....46
SM-28. Pending raises only decrease when triggered....47
SM-29. State transitions work as documented....47
SM-30. Resetting triggers works as documented....48
SM-31. Requesting raises works as documented....49
SM-32. Changing numPendingRaises causes state change....50
SM-33. Triggering behaves as documented....50
SM-34. Only the queued config update can be finalized....51
SM-35. Config updates are reset when necessary....51
SM-36. Config updates are restricted....52
SM-37. Config updates are checked properly....52
SM-38. Updating SharedSafetyModule is restricted....53
SM-39. Number of controllers is limited....53
SM-40. Queued SharedSafetyModule is the right one....54
SM-41. Raising scales pending withdrawals....55
SM-42. Pending withdrawals are at most the deposit amount....55
SM-43. Depositing reserve assets rounds down....55
RewardsManager....56
RM-01. Config updates are restricted....56
RM-02. Depositing assets updates internal accounting....57
RM-03. Asset pool amount tracks actual balance....57
RM-04. Undripped rewards only increase with balance....58
RM-05. Claiming rewards also drips rewards....59

RM-06. Rewards drip only when the manager is active.....60
RM-07. Claiming rewards transfers the correct amounts.....61
RM-08. Claiming rewards claims the full amount.....61
RM-09. Receipt token supply matches stake pool amount.....62
RM-10. Index snapshots can only increase.....62
RM-11. Each pool can only be claimed once in one call.....62
RM-12. Index snapshots for user rewards are at most those of claimable rewards.....63
RM-13. Stake pool get rewards based on weight.....63
Euler integration.....65
EI-01. Liquidator is only called in the right state.....65
EI-02. EVault liquidation do not change the state.....65
EI-03. Raise only on bad debt.....66
EI-04. Always take funds from the reserve pool first.....66
Module controllers.....66
MC-01. Only the owner can raise.....66
Disclaimer.....68
About Certora.....68

Project Summary

Project Scope

Project Name	Repository (link)	First Commit Hash	Latest Commit Hash	Platform
Safety Module (SM)	Repo Link	0367798a	3dae8b8	EVM
Rewards Manager (RM)	Repo Link	f86cec73	d208f83f	EVM
Euler integration contracts (EI)	Repo Link	b8eb758	d2e98709	EVM
(peripheral repo) Controllers	Repo Link	408d06c	22194ec2	EVM
(peripheral repo) Raise Strategies	Repo Link	6590429	6590429	EVM
(peripheral repo): Router	Repo Link	c7171c58	9582bbfd	EVM

Project Overview

This document describes the specification and verification of **Cozy Finance** using the Certora Prover. The work was undertaken from **October 6th** to **December 8th 2025**

The following contract list is included in our scope:

SafetyModule

- src/CozySafetyModuleManager.sol
- src/SafetyModule.sol
- src/SafetyModuleFactory.sol

- src/lib/Configurator.sol
- src/lib/ConfiguratorLib.sol
- src/lib/CozyMath.sol
- src/lib/Depositor.sol
- src/lib/FeesHandler.sol
- src/lib/Governable.sol
- src/lib/Ownable.sol
- src/lib/RaiseHandler.sol
- src/lib/Redeemer.sol
- src/lib/RedemptionLib.sol
- src/lib/SafetyModuleBaseStorage.sol
- src/lib/SafetyModuleCalculationsLib.sol
- src/lib/SafetyModuleCommon.sol
- src/lib/SafetyModuleInspector.sol
- src/lib/SafetyModuleStates.sol
- src/lib/StateChanger.sol
- src/lib/StateTransitionsLib.sol

RewardsManager

- src/CozyManager.sol
- src/RewardsManager.sol
- src/RewardsManagerFactory.sol
- src/StkReceiptToken.sol
- src/lib/Configurator.sol
- src/lib/ConfiguratorLib.sol
- src/lib/Depositor.sol
- src/lib/RewardsDistributor.sol
- src/lib/RewardsManagerBaseStorage.sol
- src/lib/RewardsManagerCommon.sol
- src/lib/RewardsManagerInspector.sol
- src/lib/RewardsManagerStates.sol
- src/lib/RewardsMathLib.sol
- src/lib/Staker.sol
- src/lib/StateChanger.sol
- src/lib/Withdrawer.sol

Euler integration contracts

- src/CozyLiquidationHandler.sol
- src/CozyLiquidator.sol
- src/CozyLiquidatorFactory.sol
- src/CozyLiquidatorManager.sol
- src/EulerTrancheRaiseStrategy.sol
- src/FeeReceiver.sol

Controllers (peripheral repo)

- src/OwnableController.sol
- src/OwnableControllerFactory.sol
- src/lib/Ownable.sol

Raise Strategies (peripheral repo)

- src/BasicRaiseStrategy.sol

Router (peripheral repo)

- src/CozyRouter.sol
- src/CozyRouterAvax.sol
- src/lib/router/ControllerDeploymentHelpers.sol
- src/lib/router/CozyRouterCommon.sol
- src/lib/router/DripModelDeploymentHelpers.sol
- src/lib/router/EulerVaultHelpers.sol
- src/lib/router/MerklHelpers.sol
- src/lib/router/RewardsManagerDeploymentHelpers.sol
- src/lib/router/SafetyModuleActions.sol
- src/lib/router/SafetyModuleDeploymentHelpers.sol
- src/lib/router/StEthTokenHelpers.sol
- src/lib/router/TokenHelpers.sol
- src/lib/router/WethTokenHelpers.sol

The Certora Prover demonstrated that the implementation of the **Solidity** contracts above is correct with respect to the formal rules written by the Certora team. During the verification process, the Certora team discovered bugs in the Solidity contracts code, as listed on the following pages.

Protocol Overview

Cozy Finance is a modular DeFi protection protocol whose core component, the **Cozy Safety Module (CSM)**, allows protocols to deploy dedicated protection pools that hold reserve assets used to reimburse users in the event of predefined loss conditions such as hacks or exploits. Actors include **Module Creators** (protocol teams configuring coverage rules), **Suppliers** (liquidity providers depositing assets into the protection pool in exchange for yield and risk exposure), and **Protected Users** whose losses may be compensated when a trigger fires; the **Controller** enforces the module's eligibility and payout logic, while the **RewardsManager** enables incentive programs for suppliers. Cozy also integrates with external systems such as Euler's Vault Kit, allowing CSM-backed protection to shield lenders from bad debt and enhance the safety of external lending markets.

The **CSM** architecture is modular and deterministic: modules define qualifying events, payout logic, and asset flows, enabling flexible integration with other DeFi systems. Overall, **Cozy** aims to reduce systemic DeFi risk with a transparent, automated insurance layer while aligning incentives between capital providers, integrators, and end users.

Threat and Security Overview

1. Solvency & Balance-Sheet Consistency

Affected Components: SafetyModule, RewardsManager, Module Raise, Euler integration paths, reserve/fee pools
Risk Description: The system becomes unsafe if internal accounting (deposits, claims, reserves, fees, rewards) diverges from real ERC20 balances or if the protocol ever promises more than it holds. This includes stuck funds from broken withdrawal paths, "phantom" rewards, or reserves that can be drawn down incorrectly.

Key considerations include:

- **Token-delta anchoring:** State updates should reflect actual `balanceOf` deltas (including transfer-fee/rebasing edge cases).
- **No overcommitment:** Aggregate internal claims (pending + available + fees + rewards) must never exceed actual balances.
- **Non-negativity:** Per-asset internal balances must remain ≥ 0 under all flows (deposit/withdraw/raise/slash/claim).
- **Fee ring-fencing:** Fee claims must be bounded and never draw from user principal.

Relevant Rules:

- SM-01 : Token balances are at least what has been accounted for
- SM-02 : Accounting for deposits works correctly
- SM-44 : Pending withdrawals are at most the deposit amount
- SM-05 : Reserve pool accounting corresponds to actual balance
- SM-17 : Assets amounts follow the SafetyModules balance
- SM-27 : Claimed fees are at most the accumulated fees
- RM-02 : Depositing assets updates internal accounting
- RM-03 : Asset pool amount tracks actual balance

Impact: Critical. Violations can cause insolvency, failed withdrawals/redemptions, or permanent loss/lock of funds.

2. Fair Value, Pro-Rata Redemptions & Loss Socialization

Affected Components: SafetyModule, RewardsManager (distribution), Module Raise (allocation), withdrawal/redemption queues
Risk Description: Users must never extract more than their fair share (or avoid their share of losses) due to conversion math, rounding, ordering, or partial application of haircuts. This includes redemption arithmetic, proportional slashing/insurance outcomes, and strategy-based distribution that must treat claim classes consistently.

Key considerations include:

- **Pro-rata correctness:** Redemptions/claims must align with share burn and exchange-rate math, with clearly defined rounding behavior.
- **No preferential exit lanes:** Slashing/insurance must haircut all claims consistently (pending + available) so timing cannot be exploited.
- **Strategy allocation integrity:** Any distribution/raise allocation must match intended proportions and not create cross-class dilution.

Relevant Rules:

- SM-15 : Redemptions hold the correct scaling factors
- SM-16 : Pending withdrawals are ignored when redeeming
- SM-20 : Calling trigger enters the triggered state
- SM-21 : Redeeming transfers the expected amount of assets
- SM-28: Pending raises only decrease when triggered
- SM-32: Requesting raises works as documented
- SM-44 : Pending withdrawals are at most the deposit amount
- RM-07 : Claiming rewards transfers the correct amounts

Impact: High to Critical. Breaks fairness, enables draining via rounding/timing games, and can escalate into insolvency.

3. Authorization, Governance Binding & State-Machine Safety

Affected Components: SafetyModule, Module Controller, Module Raise, governance/config update flows, Euler entrypoints
Risk Description: If privileged actions (trigger, raise, pause, config updates, dripping, liquidation-related flows) can be called by the wrong actor or in the wrong system stage,

attackers can force unsafe transitions, bypass protections, or execute operations with unapproved parameters.

Key considerations include:

- **Strict RBAC:** Controllers vs owner/governance/manager roles must be enforced everywhere (defense-in-depth).
- **State gating:** Actions like reward/fee dripping and sensitive settlement logic must only occur in intended stages (e.g., ACTIVE-only).
- **Governance integrity:** Config changes must apply exactly the parameters that were approved (hash binding) and respect delays.
- **Integration caller validation:** External protocol callers (e.g., Euler eVault) must not gain unintended authority.

Relevant Rules:

- SM-04 : Fees only drip when the SafetyModule is active
- SM-07 : Only authorized callers can unpause
- SM-13 : Authorized caller can pause
- SM-14 : Claimed fees are transferred to the owner
- SM-22 : Only the owner can complete a redemption
- SM-23 : Only the manager can claim with a custom drip model
- SM-35 : Only the queued config update can be finalized
- SM-37 : Config updates are restricted
- SM-39 : Finalizing config updates is restricted
- EI-02 : EVault liquidation do not change the state
- MC-01 : Only the owner can raise

Impact: Critical. Common root cause of catastrophic failures: forced raises, unauthorized fund movement, governance bypass, and state corruption.

4. Time Locks, Delayed Actions & Signature / Replay Safety

Affected Components: SafetyModule, Module Controller (if signatures delegated), any delayed redemption/settlement paths
Risk Description: Time-based protections and signature-gated paths can be bypassed if delays aren't enforced, signatures can be replayed, or authorization isn't tightly bound to the exact

action and parameters. This can enable early execution, double-spend behavior, or unauthorized completion of user actions.

Key considerations include:

- **Delay enforcement:** Completion must not be possible before `request.timestamp + delay`.
- **Signature scope:** EIP-712 typed data must bind action type, parameters, contract, chainId, and (if used) deadlines.
- **Single-use nonces:** Every signature must be one-time use; prevent cross-function replay.

Relevant Rules:

- SM-35 : Only the queued config update can be finalized
- SM-26 : Only complete redemptions after the delay
- SM-09: Delegated redemption can not be replayed

Impact: High to Critical. Can directly enable theft (replay), bypass protections, or break withdrawal guarantees.

5. Specification Consistency: Previews, Execution & Edge-Case Equivalence

Affected Components: SafetyModule, RewardsManager, all preview/quote functions and their state-changing counterparts
Risk Description: If preview/quote functions diverge from execution behavior, users and integrators can be misled into unsafe actions, and attackers can exploit mismatches (especially around rounding, gating, and low-liquidity edge cases). This category covers "spec drift" between read paths and write paths.

Key considerations include:

- **Preview parity:** `previewX(a)` should reflect `X(a)` under the same state and inputs.
- **Edge cases:** Zero/tiny inputs, rounding boundaries, depleted pools, post-slash states must behave consistently.
- **Stage parity:** If execution is blocked outside ACTIVE, preview should reflect that reality (or explicitly encode the same assumptions).

Relevant Rules:

- SM-21 : Redeeming transfers the expected amount of assets
- RM-07 : Claiming rewards transfers the correct amounts

Impact: Medium to High. Often creates MEV/integration exploits and user losses; can become critical when it enables systematic extraction.

6. External Integration & Reentrancy-Style Surface Risks (Validated Entry Only)

Affected Components: Euler integration paths (eVault caller), SafetyModule entrypoints reachable via integrations, Module Raise/trigger paths reachable indirectly
Risk Description: External protocol integrations expand the attack surface: unexpected call contexts, callback-like behavior, or partially validated entry conditions can cause unintended state changes, reentrancy-like effects, or execution of sensitive operations from an untrusted caller.

Key considerations include:

- **Validated entry conditions:** Calls from integration counterparties must be strictly matched to allowed call patterns.
- **No unintended side effects:** If an integration caller is not on the approved path, functions must revert or return without state mutation.
- **Path isolation:** Sensitive raise/trigger/liquidation resolution should only occur through explicitly allowed, audited paths.

Relevant Rules:

- EI-01 : Liquidator is only called in the right state
- EI-03 : Raise only on bad debt
- EI-04 : Always take funds from the reserve pool first

Impact: High to Critical. Integration boundary bugs are frequent exploit vectors and can lead to immediate, systemic fund loss.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	2	2	1
Low	-	-	-
Informational	5	5	1
Total	7	7	2

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Detailed Findings

ID	Title	Severity	Status
M-01	Rewards are not dripped upon stake token transfers	Medium	Acknowledged
M-02	SafetyModule can add a controller that is not registered on the manager	Medium	Fixed

Medium Severity Issues

M-01 Rewards are not dripped upon stake token transfers		
Severity: **Medium**	Impact: **Medium**	Likelihood: **Medium**
Files: RewardsDistributor.sol	Status: Acknowledged	

Description: Accounts depositing into the RewardsManager receive StkReceiptTokens, which accrue rewards. The StkReceiptTokens can be transferred and each transfer executes a logic that makes sure that rewards are allocated first for each user, before updating their balances. However dripRewards() is not called and only the already accounted snapshots are used , meaning that all pending non-dripped rewards that should belong to from would now belong to the to address

```javascript
/// @dev Updates the user's rewards before transferring the stkReceiptTokens by calling into the rewards manager.
  function transfer(address to_, uint256 amount_) public override returns (bool) {
    IRewardsManager(module).updateUserRewardsForStkReceiptTokenTransfer(msg.sender, to_);
    return super.transfer(to_, amount_);
  }
```

This issue was uncovered during the following prover run.

Recommendations: Call dripRewards() when transferring stake tokens

Customer's response: Acknowledged - *"The rewards accrual behavior on transfers is intentional."*

Fix Review: Acknowledged

M-02 SafetyModule can add a controller that is not registered on the manager

Severity: **Medium**	Impact: **Medium**	Likelihood: **Medium**
Files: ConfiguratorLib.sol	Status: Fixed	

Description: There is an edge case when queuing a config update to add/remove controllers in a SafetyModule that allows having a controller address added to the controllers_ state array, without having it registered on the manager contract

```javascript
if (exists_ && !controllerData_[controller_].exists) {
        controllers_.push(controller_);
      } else if (!exists_ && controllerData_[controller_].exists) {
        findAndRemoveController(controllers_, controller_);
        manager_.unregisterSafetyModuleController(controller_);
      }
```

Here is an example describing the issue:

- In array controllers_= [ctr1,ctr2] - contains 2 registered controllers.
- An update is queued where ctr2 is provided 2 times -[{exists:false}, {exists:true}]

What happens inside the applyConfigUpdates() is that in the first loop ctr2 is removed and unregistered. In the 2nd loop it is re-added to controllers_ but it is not registered again, since this happens in stage 1 of the config update process

Recommendations: When queueing updates in updateConfigs(), make sure to check for duplicate entries, or alternatively check for re-registration in the finalization stage

Customer's response: Fixed in commit 69ef733

Fix Review: Fixed

Informational Issues

I-01. Module can be raised with non-reset expired triggers

Description: Currently triggerIds can expire and once expired, they can be reset through resetTriggerEvent() , which makes them un-raisable. At the same time the requestRaise() function does not check if the provided triggerId is expired and allows raising triggers that are expired and but not reset yet

This issue was uncovered during the following prover run.

Recommendation: Check that the trigger is not expired

Customer's response: Acknowledged - " *The* requestRaise() *behavior is intentional. We primarily added the reset trigger event functionality as a failsafe to handle the edge case where a SM is triggered by a controller, but the controller is unable to execute a raise (e.g. it has a bug, etc.). That leaves the SM in a permanently TRIGGERED state, which is undesirable. So, you have the option to reset a trigger event post-expiry, but in case no one decides to reset it post-expiry, it is allowed to be raised.*"

Fix Review: Acknowledged

I-02. Some config updates can be queued while module is triggered

Description: Initiating most of the updates in the Configurator contract revert if state is TRIGGERED, except for setProposedSharedSafetyModule() which does not check for the status and allows the proposedSharedSafetyModule_ parameter to be set even when triggered.

This issue was uncovered during the following prover run.

Recommendation: Consider checking for triggered state

Customer's response: Acknowledged - "*Not checking for* TRIGGERED *in* setProposedSharedSafetyModule() *is intended. The proposal does not change anything about the* SM*, until the shared* SM *is actually queued via* queueSharedSafetyModule()*, where we do indeed check that the* SM *is not* TRIGGERED*. Checking at queue time ensures that the config update delay clock does not start while the* SM *is* TRIGGERED*, so that depositors are unable to withdraw before the shared SM update is finalized.*"

Fix Review: Acknowledged

I-03.A safetyModule can register an unlimited number of controllers in the manager

Description: When a safetyModule wants to add new controllers this happens in 2 stages - the first one queues a delayed update and the second one finalizes it. In the first stage the current length of controllers_ is taken, summed with the new controllers count and checked against the limit. However since the controllers_ array only updates upon finalization (2nd stage), but controllers are always registered in the 1st stage, the following can happen:

- max = 10 / len(controllers_) = 7 / newControllerstoAdd = 3 -> the 3 new controllers would get registered into the manager. Then I can call updateConfigs() once again with another set of 3 new controllers, which would pass validation and they would also be registered into the Manager

This will not violate the max length of controllers_ , but it allows infinite registration of controllers into the manager (above the max)

Recommendation: Validate the number of registered controllers into the manager

Customer's response: Acknowledged - "*This is not a problem. We have a maxControllers argument to function as a loose upper bound on the size of the controllers_ array. This ensures we do not run into any out-of-gas errors in the protocol in case you need to iterate over the controllers_ array. Registering controllers beyond the maxControllers param in the Manager is OK.*"

Fix Review: Acknowledged

I-04. Controller registration can be front-runned and DOSed for a SafetyModule by another one

Description: When a safetyModule wants to add new controllers this happens in 2 stages - the first one queues a delayed update and the second one finalizes it. In the first stage the controllers are registered into the manager and a safetyModule can register any arbitrary address as its controller.

This creates a low likelihood griefing scenario where someone can deploy a dummy SafetyModule to front-run actual modules and register their controllers on the dummy one. This could be problematic, since it requires the re-deployment of new controllers for the honest modules

Recommendation: Given the lack of economical gain from the above case, it is very unlikely that an actor would be incentivized to exploit it. But the team should be aware of it

Customer's response: Acknowledged - "*The extreme edge case is that someone spends resources to monitor the mempool and frontruns before you are able to atomically deploy the controller + queue a config update. We decided this is OK. It seems extremely unlikely someone would carry out this DOS attack given the cost and no benefit.*"

Fix Review: Acknowledged

I-05. Rewards can round down to 0 for small amounts when adding pools in claimRewards

Description: The two step process of calculating accrued rewards inside the claimRewards() flow of RewardsDistributor introduces an edge case, where very small index changes could yield 0 rewards under some configurations.

The _claimRewards() internal function has the following logic for calculating user accrued rewards on newly added pools

```JavaScript
function _claimRewards(ClaimRewardsArgs memory args_, ClaimRewardsPoolData[] memory
claimRewardsPoolData_)
    internal
    override
  {
....

for (uint16 i = numUserRewardAssets_; i < numRewardAssets_; i++) {
      userRewards_.push(
        _previewAddUserRewardsData(claimRewardsData_.userStkReceiptTokenBalance,
claimableRewards_[i].indexSnapshot)
      );
    }

....
for (uint256 i; i < claimRewardsPoolData_.length; i++) {
....

 (ClaimableRewardsData memory newClaimableRewardsData_,) = _previewNextClaimableRewardsData(
          claimableRewards_[rewardPoolId_],
          rewardPool_.cumulativeDrippedRewards,
          claimRewardsData_.stkReceiptTokenSupply,
          claimRewardsData_.rewardsWeight
        );
...
}
....
}
```

First it loops through all registered reward pools and if some of them are not inside the userRewards_ array, accruedRewards are updated based on the current index of stakePoolId (without dripping the reward pools). After that reward pools are dripped and stakePoolId is

updated to the latest dripped index. Again accruedRewards are updated based on the increase of indexSnapshot from first step

Since index calculation is done in 2 steps (divided 2 times) this creates a scenario where small index changes can round down to 0, instead of producing a non-zero amount if they were calculated at once. Here is a concrete scenario:

```javascript
// there are 2 pools and the second one is new to the user
rewardPools = [pool1, pool2]
claimableRewards[stakePool][pool2].indexSnapshot = 2
// this is the index we would get after dripping pool2
claimableRewards[stakePool][pool2].latestIndexSnapshot = 7
userstReceiptTokenBalance =  199999999999999999e18
// 0 on newly added pools
userRewards_[pool2].indexSnapshot = 0

//calculating index change at once - previewClaimableRewards()
// 7 - latest index after dripping
// 0 - initial index for user on new pool
userstReceiptTokenBalance * (7-0) / 1e18**2 => 1

//calculating index change in 2 steps - claimRewards()
 //1 step - between stake pool latest index(2) and user initial index(0)
userstReceiptTokenBalance * (2-0) / 1e18**2 => 0
// 2 step - between latest stake pool dripped index(7) and latest on user(2)
userstReceiptTokenBalance * (7-2) / 1e18**2 -> 0
```

As you can see although the index difference is 7 in both cases, the second scenario that calculates them in 2 steps, produces a final reward amount of 0 instead of 1

The above scenario is an edge case one and is only possible on specific configurations where the index change is really small and underlying tokens are low value and have big supply. As result also the preview (1 step) function would return a different value than claimRewards()(2 step)

This issue was uncovered during the following prover run.

Recommendations: The issue can be fixed by calculating the accrued rewards based on the total index change instead of splitting it in two stages when claiming rewards

Customer's response: Fixed in commit 44d03785

Fix Review: Fixed

Formal Verification

Verification Methodology - Solidity

We performed verification of **Cozy Finance** protocol using the Certora verification tool which is based on Satisfiability Modulo Theories (SMT). In short, the Certora verification tool works by compiling formal specifications written in the Certora Verification Language (CVL) and **Cozy Finance**'s implementation source code written in Solidity.
More information about Certora's tooling can be found in the Certora Technology Whitepaper.

If a property is verified with this methodology it means the specification in CVL holds for all possible inputs. However specifications must introduce assumptions to rule out situations which are impossible in realistic scenarios (e.g. to specify the valid range for an input parameter). Additionally, SMT-based verification is notoriously computationally difficult. As a result, we introduce overapproximations (replacing real computations with broader ranges of values) and underapproximations (replacing real computations with fewer values) to make verification feasible.

Rules: A rule is a verification task possibly containing assumptions, calls to the relevant functionality that is symbolically executed and assertions that are verified on any resulting states from the computation.

Inductive Invariants: Inductive invariants are proved by induction on the structure of a smart contract. We use constructors as a base case, and consider all other (relevant) externally callable functions that can change the storage as step cases.
Specifically, to prove the base case, we show that a property holds in any resulting state after a symbolic call to the respective constructor. For proving step cases, we generally assume a state where the invariant holds (induction hypothesis), symbolically execute the functionality under investigation, and prove that after this computation any resulting state satisfies the invariant.

Verification Notations

Formally Verified	The rule is verified for every state of the contract(s), under the assumptions of the scope/requirements in the rule.
Formally Verified After Fix	The rule was violated due to an issue in the code and was successfully verified after fixing the issue
Violated	A counter-example exists that violates one of the assertions of the rule.

General Assumptions and Simplifications

1. We enable the prover to check setup with a SharedSafetyModule and without one in one go. The SharedSafetyModule is in the scene and all calls are optimistically dispatched. All calls to the SharedSafetyModule are protected by checks whether its address in the SharedSafety is zero (checked by SM-24). The prover can always choose the zero branch (without SharedSafetyModule) or the non-zero branch (with SharedSafetyModule).
2. The prover has some special treatment for cloning contracts, as is done in the different factories. To model this correctly, we need to have unique bytecode data in the cloneDeterministic method. We thus added a separate cloneDeterministic_SM method for the SafetyModuleFactory.
3. All token classes used in this project are assumed to not exhibit overflow or underflows. We made this transparent to the prover by adding explicit checks to all relevant ERC20 methods. To make this work, we made transfer and transferFrom virtual in the StkReceiptToken contract.
4. We restricted the _claimRewards method to allow for a single claim only. This made this part of the code tractable for the prover. We manually checked for any possible interactions between multiple claims.
5. No actual implementation was used for the DripModel. We instead considered any possible DripModel that returns zero if no time passed since the last drip time, grows monotonically over time and is always below one (or rather WAD). This was modelled in snippet_DripModel.spec.

6. All functions used in the context of hashing (domain separators, packing strings, salt, hashing custom struct) were ignored and allowed to behave arbitrarily. We used NONDET summaries.
7. For each class of tokens, we assumed the presence of two separate tokens. This covers scenarios where multiple pools might interact, but are still tractable for the prover. By manual inspection, we did not identify anything where more than two tokens would be necessary to obtain a general property for arbitrarily many tokens.
   In the RewardsManager, we additionally assumed that there are at most two reward pools and at most two stake pools; furthermore, the stake receipt tokens are distinct for different stake pools.
8. The implementation of negLn and expNeg from RewardsMathLib was not inspected. We replaced both by two arbitrary functions that were only assumed to be monotonically decreasing.

Formal Verification Properties Overview

ID	Title	Impact	Status
SafetyModule			
SM-01	Token balances are at least what has been accounted for	Prevents the contract from overestimating its reserves; violations could cause insolvency or under-collateralization.	Verified
SM-02	Accounting for deposits works correctly	Ensures deposits are correctly credited; violations could result in user funds not being properly accounted and potential losses.	Verified
SM-03	State is triggered exactly when raises are pending	Prevents state inconsistencies that could block or erroneously enable sensitive operations (like asset withdrawal or emergency response).	Verified
SM-04	Fees only drip when the SafetyModule is active	Unauthorized/mis-timed fee drip or deposit changes could result in incorrect fund distribution or fee drain.	Verified
SM-05	Reserve pool accounting corresponds to actual balance	Prevents accounting mismatches that could enable funds to be lost, stolen, or manipulated.	Verified
SM-06	Only drip methods drip	Restricts fee movement to authorized methods, mitigating risk of unauthorized fee manipulation.	Verified
SM-07	Only authorized callers can unpause	Prevents unauthorized re-activation after a pause (which may have been enacted for a security reason).	Verified

SM-08	Accounting for redemption completion is correct	Ensures that the redemption process transfers funds as documented.	Verified
SM-09	Delegated redemption can not be replayed	A violation could let attackers claim multiple withdrawals from a single redemption.	Verified
SM-10	Only authorized users can skim	Prevents unauthorized asset transfers and guarantees access control for special skim operations.	Verified
SM-11	Redemption fees are at most the redeemed amount	Prevents fee logic bugs from over-charging/redirection of user funds.	Verified
SM-12	Can not redeem when triggered	Prevents redeeming while funds should be locked for a potential upcoming raise.	Verified
SM-13	Authorized caller can pause	Prevents unauthorized denial-of-service or circumvention of safety mechanisms.	Verified
SM-14	Claimed fees are transferred to the owner	Prevents fees from being claimed by unauthorized parties, avoiding fee theft or incorrect distribution.	Verified
SM-15	Redemptions hold the correct scaling factors	Ensures that the correct scaling factors are stored to allow for proper accounting when redeeming after a raise.	Verified
SM-16	Pending withdrawals are ignored when redeeming	Checks correct accounting math when redemption are already queued.	Verified
SM-17	Assets amounts follow the	Detects inconsistent asset movement; a violation could mask or cause asset loss and	Verified

	SafetyModules balance	accounting bugs.	
SM-18	Unpause leaves the pause state	Ensures the contract does not become ACTIVE prematurely or remain paused/triggered unnecessarily, protecting user and system integrity.	Verified
SM-19	Only claim methods can claim fees	Prevents unauthorized draining of contract asset balances by restricting balance reductions to legitimate claims.	Verified
SM-20	Calling trigger enters the triggered state	Prevents unauthorized state transitions that could put assets or system logic at risk.	Verified
SM-21	Redeeming transfers the expected amount of assets	Ensures that completing a redemption transfers funds as documented.	Verified
SM-22	Only the owner can complete a redemption	Prevents unauthorized users to extract funds from a pending redemption.	Verified
SM-23	Only the manager can claim with a custom drip model	Restricts advanced fee claim operations to trusted entities only, avoiding fee misappropriation.	Verified
SM-24	Never call the SharedSafetyModule when it is zero	Avoids reverting behavior by never calling a null address. Mostly to show the prover setup is sane and sound.	Verified
SM-25	An unauthorized caller can not pause	Prevents denial-of-service by untrusted accounts who might pause operations maliciously.	Verified
SM-26	Only complete redemptions after	Prevents premature completion/deletion of redemptions, ensuring time-based release is	Verified

	the delay	respected.	
SM-27	Claimed fees are at most the accumulated fees	Prevents over-disbursement of fees, preserving contract reserves.	Verified
SM-28	Pending raises only decrease when triggered	Violations could put the contract into an inconsistent state and potentially cause asset mismanagement.	Verified
SM-29	State transitions work as documented	A violation could result in unauthorized changes to sensitive state.	Verified
SM-30	Resetting triggers works as documented	Ensure that resetting triggers only works in appropriate circumstances and updates the state accordingly.	Verified
SM-31	Requesting raises works as documented	Violations could lead to draining funds or allow for raises in unexpected situations.	Verified
SM-32	Changing numPendingRaises causes state change	Prevents incorrect state after raise additions/removals, reducing risk of stuck or unsafe contract state.	Verified
SM-33	Triggering behaves as documented	Prevents abuse of the raise system and ensures accountability and correct event workflow.	Verified
SM-34	Only the queued config update can be finalized	Violations could allow for introducing arbitrary configuration.	Verified
SM-35	Config updates are reset when necessary	Ensures that pending config updates are removed to avoid inconsistent configuration data.	Verified

SM-36	Config updates are restricted	Prevents changes to already existing reserve pools and restricts config updates to a specific time window.	Verified
SM-37	Config updates are checked properly	Prevents the introduction of invalid or hazardous configurations.	Verified
SM-38	Updating SharedSafetyModule is restricted	Prevents erroneous updates of the SharedSafetyModule.	Verified
SM-39	Number of controllers is limited	Enforces a limit on the number of registered controllers.	Verified
SM-40	Queued SharedSafetyModule is the right one	Prevents incorrect updates of the SharedSafetyModule.	Verified
SM-41	Raising scales pending withdrawals	Prevents incorrect scaling of the pending withdrawals amount when raising.	Verified
SM-42	Pending withdrawals are at most the deposit amount	Violations would allow more withdrawals than deposited, enabling asset theft or double-spending.	Verified
RewardsManager			
RM-01	Config updates are restricted	Prevents modifications of already registered pools and assets. Checks the rewards weights to add up properly.	Verified
RM-02	Depositing assets updates internal accounting	Checks that assets are moved correctly, prevents accounting errors when depositing.	Verified
RM-03	Asset pool amount tracks actual	Avoids inconsistencies between the internal accounting and the actual balances.	Verified

	balance		
RM-04	Undripped rewards only increase with balance	Avoids erroneous accounting of undripped rewards, which could lead to dripping more rewards than planned.	Verified
RM-05	Claiming rewards also drips rewards	Checks that rewards first drip before they are claimed, avoiding edge cases in the handling of undripped rewards.	Verified
RM-06	Rewards drip only when the manager is active	Enforces that no rewards drip when the manager is not active.	Verified
RM-07	Claiming rewards transfers the correct amounts	Checks that assets are moved correctly when claiming rewards. Violations could point to inconsistent reward updates.	Verified after fix
RM-08	Claiming rewards claims the full amount	Ensures convenient semantics when claiming rewards that claim all available rewards.	Verified
RM-09	Receipt token supply matches stake pool amount	Violations would show inconsistencies in the accounting of the stake receipt tokens.	Verified
RM-10	Index snapshots can only increase	Decreasing index snapshot would point to issues in the reward calculation logic.	Verified
RM-11	Each pool can only be claimed once in one call	Checks a basic assumption when claiming rewards, simplifying the code and logic of the respective code.	Verified
RM-12	Index snapshots for user rewards are at most those of claimable rewards	Violations would allow users to claim rewards that are not available, or at least have not dripped yet.	Verified

RM-13	Stake pool get rewards based on weight	Verifies that the stake pools are rewarded correctly according to their weight.	Verified
Euler integration			
EI-01	Liquidator is only called in the right state	The integration of the liquidator with the EVault follows the specified call patterns.	Verified
EI-02	EVault liquidation do not change the state	Any call to liquidation coming from the EVault has no tangible effect.	Verified
EI-03	Raise only on bad debt	Funds are only raised from the SafetyModule if the EVault itself can not satisfy any bad debt.	Verified
EI-04	Always take funds from the reserve pool first	The tranching raise strategy works as specified, using the fee pool first and only then using the lender pool.	Verified
Module controllers			
MC-01	Only the owner can raise	Only the recognized owner can raise funds from a SafetyModule via the controller.	Verified

Detailed Properties

SafetyModule

Module General Assumptions

- The block timestamp is assumed to fit within 128 bits.
- We assume all tokens to be reasonable ERC20 implementations and not exhibit any overflows or underflows.

Module Properties

SM-01. Token balances are at least what has been accounted for	
Status: Verified	Ensures solvency of the contract in the managed asset pool.

Rule Name	Status	Description	Link to rule report
accountedForReserveLEQReserveBalance	Verified	*Ensures that the accounted amount in a reserve pool never exceeds the actual safety balance. This prevents accounting errors where the contract thinks it has more assets than it actually holds.*	*Report*

SM-02. Accounting for deposits works correctly	
Status: Verified	The deposit methods do the expected accounting.

Rule Name	Status	Description	Link to rule report
depositIncreasesReserve	Verified	*Checks that depositReserveAssets updates the internal accounting correctly. The contracts balance in the asset should increase just as the reserve amount, the receipt token amount should change as*	*Summarized Report*

		shown by the preview function and the deposit amount should increase (minus fees).	
depositWithoutTransferIncreasesReserve	Verified	*Checks that depositReserveAssetsWithoutTransfer updates the internal accounting correctly. The contracts balance in the asset should not change, the receipt token amount should change as shown by the preview function and the deposit amount should increase (minus fees).*	

SM-03. State is triggered exactly when raises are pending

Status: Verified	The SafetyModule state properly follows whether there are pending raises.

Rule Name	Status	Description	Link to rule report
nonZeroPendingRaisesImpliesTriggeredOrPaused	Verified	*Ensures that if there are any pending raises, the safety module must be in either TRIGGERED or PAUSED state. This prevents the contract from being in ACTIVE state when there are unresolved raise requests.*	Summarized Report
zeroPendingRaisesImpliesActiveOrPaused	Verified	*Ensures that if there are no pending raises, the safety module must be in either ACTIVE or PAUSED state. This prevents the contract from being in TRIGGERED state when there are no active raise requests.*	

SM-04. Fees only drip when the SafetyModule is active

Status: Verified	Fees only drip when the SafetyModule is active.

Rule Name	Status	Description	Link to rule report

onlyDripWhenActive	Verified	*When the safety module is not active, the deposit amount should not change.*	*Summarized Report*
onlyChangeLastFeesDripTimeWhenActive	Verified	*When the safety module is not active, the last fees drip time should not change. The only exception is the syncLastFeesDripTime function.*	

SM-05. Reserve pool accounting corresponds to actual balance

Status: Verified	The internal accounting is consistent with the SafetyModules actual balance in the reserve pool.

Rule Name	Status	Description	Link to rule report
reservePoolReflectsBalance	Verified	*Verifies that changes to the reserve pool's deposit amounts and pending withdrawals are correctly reflected in the actual balance. When deposits decrease or pending withdrawals are completed, the balance change must match the corresponding amount changes.*	*Report*

SM-06. Only drip methods drip

Status: Verified	Only the dedicated drip methods drip fees.

Rule Name	Status	Description	Link to rule report
onlyDripMethodsDrip	Verified	*Checks that the fee amount and the deposit amount never changes, unless by calling a drip method (dripFees or dripFeesFromReservePool).*	*Report*

SM-07. Only authorized callers can unpause

Status: Verified	Only authorized callers can call unpause.

Rule Name	Status	Description	Link to rule report
unauthorizedCallerCannotUnpause	Verified	*Ensures that only authorized parties (owner, pauser, or cozySafetyModuleManager) can unpause the contract. Any call to unpause from an unauthorized address must revert.*	Report

SM-08. Account for redemption completion is correct

Status: Verified	The different ways to complete a redemption all update the internal accounting correctly.

Rule Name	Status	Description	Link to rule report
correctAccountForCompleteRedemptionBySig	Verified	*Calling completeRedemptionBySig correctly updates the depositAmount, pendingWithdrawalsAmount, and asset pool amount.*	Summarized Report
correctAccountForCompleteRedemption	Verified	*Calling completeRedemption correctly updates the depositAmount, pendingWithdrawalsAmount, and asset pool amount.*	
correctAccountingForRest	Verified	*Checks that no function other than completeRedemptionBySig and completeRedemption completes a redemption. Actually checks that no redemption owner is reset to zero, which corresponds to deleting a redemption.*	

SM-09. Delegated redemption can not be replayed

Status: Verified	A delegated redemption can only be executed via completeRedemptionBySig once. This prevents replay attacks to redeem more than once.

Rule Name	Status	Description	Link to rule report
cannotRedeemWithSigTwice	Verified	*Calling completeRedemptionBySig a second time with the same id reverts.*	Report

SM-10. Only authorized users can skim

Status: Verified	Only the user set as skimReceiver can skim funds, whether by calling skim or any other single function. This does not check for a combination of functions that allow to skim; skimming is still possible by first claiming orphaned funds with depositWithoutTransfer and redeem them via redeem and completeRedeem.

Rule Name	Status	Description	Link to rule report
onlySkimReceiverCanSkim	Verified	*Only the address stored in skimReceiver can call skim. Also, skim does not change any token balance other than the contract's and the skim receiver's.*	Summarized Report
canOnlySkimWithSkim	Verified	*Only the skim function can perform a skim operation, i.e. decrease the contract's balance when its internal amount is below the actual balance. Note that this only checks for single functions, not a sequence of functions.*	

SM-11. Redemption fees are at most the redeemed amount

Status: Verified	Redemption fees are not excessive, but at most what is being redeemed.

Rule Name	Status	Description	Link to rule report
redemptionFeeSmallerThanTotalAssetAmount	Verified	*Verifies that the redemption fee is never greater than the total asset amount in a redemption. This ensures fees are always reasonable and cannot exceed the redemption value.*	Report

SM-12. Can not redeem when triggered

Status: Verified	Redemptions are impossible when the SafetyModule is triggered.

Rule Name	Status	Description	Link to rule report
cannotRedeemWhenTriggered	Verified	*The redeem function always reverts when the safety module is in triggered state.*	Report

SM-13. Authorized caller can pause

Status: Verified	An authorized caller can pause the SafetyModule.

Rule Name	Status	Description	Link to rule report
authorizedCallerCanPause	Verified	*Verifies that authorized callers (owner, pauser, or manager) can successfully pause the contract. If the contract is already paused, attempting to pause again should revert.*	Report

SM-14. Claimed fees are transferred to the owner

Status: Verified	Claimed fees are payed out only to the owner specified in the SafetyModuleManager, or to the given address on behalf of the SafetyModuleManagers owner.

Rule Name	Status	Description	Link to rule report
claimedFeesAreTransferredOnlyToOwnerA	Verified	*The first claimFees overload only changes the balances of the safety module and the given owner. All other balances remain the same, in particular the fees are not transferred to anyone but the given owner. This can only be done by the owner of the SafetyModuleManager.*	*Summarized Report*
claimedFeesAreTransferredOnlyToOwnerB	Verified	*The second claimFees overload only changes the balances of the safety module and the owner of the SafetyModuleManager. All other balances remain the same, in particular the fees are not transferred to anyone but the proper owner.*	

SM-15. Redemptions hold the correct scaling factors

Status: Verified	When a redemption is queued, the correct scaling factors from pending redemptions are stored.

Rule Name	Status	Description	Link to rule report
properRedemptionFactorsStored	Verified	*Calling redeem properly copies the current scaling factors from the reserve pool to the redemption.*	*Report*

SM-16. Pending withdrawals are ignored when redeeming

Status: Verified	When calculating the amount of assets to redeem and shares to burn, the currently pending withdrawals need to be subtracted from the total amount of assets.

Rule Name	Status	Description	Link to rule report
redeemExcludesPendingWithdrawals	Verified	*The redeem function makes sure that the pending withdrawals amount is ignored for the calculation of the total amount of tokens that can be redeemed.*	Report

SM-17. Assets amounts follow the SafetyModules balance

Status: Verified	The asset amounts in the internal accounting should always follow changes in the actual asset balance of the SafetyModule. The first exception is skim where assets are transferred out without changing the internal accounting. The second exception is depositReserveAssetsWithoutTransfer where the internal accounting is updated under the assumption that the asset transfer has already happened.

Rule Name	Status	Description	Link to rule report
poolAmountOnlyChangesWithBalance	Verified	*Ensures that changes to the accounted pool amount always match changes to the actual reserve balance. This maintains consistency between accounting and actual token balances for all methods except skim and depositReserveAssetsWithoutTransfer.*	Summarized Report
poolAmountOnlyChangesWithBalance_skim	Verified	*Verifies that skim operations correctly handle accounting: if skimming the reserve asset, the accounted amount should equal the balance after skim. The accounted amount should remain unchanged, and the balance should never increase.*	

poolAmountOnlyChangesWithBalance_depositReserveAssetsWithoutTransfer	Verified	*Verifies that depositReserveAssetsWithoutTransfer correctly updates accounting without changing the actual balance. The accounted amount should increase by the deposited amount, while the balance remains unchanged since no transfer occurs.*	

SM-18. Unpause leaves the pause state

Status: Verified	Calling unpause properly transitions from paused to either active or triggered.

Rule Name	Status	Description	Link to rule report
unpauseDoesProperTransition	Verified	*Verifies that unpause correctly transitions to TRIGGERED state if there are pending raises, or to ACTIVE state otherwise. Unpause should only succeed when the contract is in PAUSED state, otherwise it should revert.*	Report

SM-19. Only claim methods can claim fees

Status: Verified	Only the dedicated methods for claiming fees can pay out fees.

Rule Name	Status	Description	Link to rule report
onlyClaimMethodsClaim	Verified	*No methods, except the two claimFees overloads, can change the balance of the contract. Note that this rule is checked in the context of the FeesHandler contract and thus not instantiated with, e.g., the redemption methods.*	Report

SM-20. Calling trigger enters the triggered state

Status: Verified	Calling trigger correctly transitions into the triggered state. It can only be called by authorized callers.

Rule Name	Status	Description	Link to rule report
triggerDoesProperTransition	Verified	*Verifies that trigger correctly transitions to TRIGGERED state when called by an authorized controller, unless the contract is already PAUSED. Unauthorized callers should not be able to trigger the contract, and triggering while paused should not change the state.*	Report

SM-21. Redeeming transfers the expected amount of assets

Status: Verified	The redemption queued by redeem has the same asset amount and fee amount as the previewRedemption function.

Rule Name	Status	Description	Link to rule report
redeemCorrectAmount	Verified	*The redeem function behaves as one would expect from the previewRedemption function.*	Report

SM-22. Only the owner can complete a redemption

Status: Verified	A redemption can only be completed by its owner, except when completeRedemptionBySig is used.

Rule Name	Status	Description	Link to rule report

onlyOwnerCanCompleteRedemption	Verified	*Redemptions can only be completed by the proper owner. A redemption is considered to be completed if the owner is reset to zero. This does not hold for completeRedemptionBySig, which is explicitly meant to allow redeeming on behalf of the owner.*	Report

SM-23. Only the manager can claim with a custom drip model

Status: Verified	Calling claimFees with a custom drip model can call an unexpected amount of fees. Thus, only the SafetyModuleManager can do that.

Rule Name	Status	Description	Link to rule report
onlyManagerCanClaimWithCustomModel	Verified	*Only the cozySafetyModuleManager can call claimFees with a custom drip model.*	Report

SM-24. Never call the SharedSafetyModule when it is zero

Status: Verified	The SafetyModule can work with or without a SharedSafetyModule. To make the prover setup sane, verify that no call is made to the SharedSafetyModule when its address is zero. Furthermore check that the prover considers both possibilities: sharedSafetyModule being zero or non-zero.

Rule Name	Status	Description	Link to rule report
neverCallZeroSSM	Verified	*Checks that we never attempt to call into the shared safety module when the sharedSafetyModule address is zero. This should be irrelevant in the wild as a call to zero reverts, but it makes the setup in the prover easier. With this assumption proven, we can ignore some corner cases with dispatchers.*	Report

SSM_canBeZero	Verified	*Sanity rule to check that the SharedSafetyModule can be zero.*	*Summarized Report*
SSM_canBeNonZero	Verified	*Sanity rule to check that the SharedSafetyModule can be non-zero.*	

SM-25. An unauthorized caller can not pause

Status: Verified	Only the owner, pauser or SafetyModuleManager can call pause.

Rule Name	Status	Description	Link to rule report
unauthorizedCallerCannotPause	Verified	*Ensures that only authorized callers (owner, pauser, or manager) can pause the contract. Unauthorized callers attempting to pause should cause the operation to revert.*	*Report*

SM-26. Only complete redemptions after the delay

Status: Verified	A redemption is completed only after the delay or when the SafetyModule is paused.

Rule Name	Status	Description	Link to rule report
onlyDeleteRedemptionAfterDelay	Verified	*A redemption can only be completed after the delay has passed, or when the safety module is paused.*	*Report*

SM-27. Claimed fees are at most the accumulated fees

Status: Verified	Calling claimFees pays out at most as many assets as were accumulated in the feeAmount, possibly after dripping fees.

Rule Name	Status	Description	Link to rule report
claimedFeesAreAtMostTheFeeAmountA	Verified	*The first claimFees overload transfers the fees, but not more.*	Summarized Report
claimedFeesAreAtMostTheFeeAmountB	Verified	*The second claimFees overload transfers the fees, but not more.*	

SM-28. Pending raises only decrease when triggered

Status: Verified	The global numPendingRaises counter can only decrease when the SafetyModule is triggered. Also, the global numPendingRaises counter is at most the sum of the individual per-controller numPendingRaises counters.

Rule Name	Status	Description	Link to rule report
numPendingRaisesIsConsistent	Verified	*Verifies that the sum of pending raises across controllers remains at least the global numPendingRaises after any method call. This ensures consistency between controller-level and global pending raise accounting.*	Summarized Report
numPendingRaisesOnlyDecreasesWhenTriggered	Verified	*Ensures that the number of pending raises can only decrease when the contract is in TRIGGERED state. This prevents pending raises from being cleared inappropriately when the contract is not triggered.*	

SM-29. State transitions work as documented

Status: Verified	The various state transitions work as specified in the documentation.

Rule Name	Status	Description	Link to rule report
canPauseIfActive	Verified	*Verifies that the contract can be paused when in ACTIVE state. This ensures pause functionality works correctly from the active state.*	Summarized Report
canPauseIfTriggered	Verified	*Verifies that the contract can be paused when in TRIGGERED state. This ensures pause functionality works correctly from the triggered state.*	
canUnpauseOnlyIfPaused	Verified	*Verifies that unpause can only succeed when the contract is in PAUSED state. This ensures unpause operations are only valid from the paused state.*	
triggerChangesStateProperly	Verified	*Verifies that trigger correctly transitions to TRIGGERED state unless the contract is already PAUSED. If paused, the state remains PAUSED; otherwise it transitions to TRIGGERED.*	

SM-30. Resetting triggers works as documented

Status: Verified	The resetTriggerEvent reverts if called with invalid data. If it works, it properly reduces the number of pending raises both globally and for that particular controller.

Rule Name	Status	Description	Link to rule report
resetTriggerEventRevertsIfNotExpired	Verified	*Verifies that resetTriggerEvent reverts when called on a trigger event that has not yet expired. This ensures only expired events can be reset.*	Summarized Report

resetTriggerEventRevertsIfNotPendingRaise	Verified	*Verifies that resetTriggerEvent reverts when the trigger event is not in PENDING_RAISE state. This ensures only pending raise events can be reset.*	
resetTriggerEventRevertsIfNotTriggered	Verified	*Verifies that resetTriggerEvent reverts when the contract is not in TRIGGERED state. This ensures reset operations can only occur during triggered state.*	
resetTriggerEventReducesNumPendingRaises	Verified	*Verifies that resetTriggerEvent correctly decreases both the global and controller-specific pending raises counts. This ensures accounting is properly maintained when trigger events are reset.*	
resetTriggerEventRevertsWhenCalledAgain	Verified	*Verifies that resetTriggerEvent reverts when called again on an already reset trigger event. This prevents double-resetting of the same event.*	

SM-31. Requesting raises works as documented

Status: Verified	The requestRaise function reverts when it is not called with a suitable trigger. If it works, it decreases numPendingRaise both globally and for the controller.

Rule Name	Status	Description	Link to rule report
resetEventIdsCannotRaise	Verified	*Verifies that requestRaise reverts when the contract is not in TRIGGERED state. This ensures raises can only be requested during the triggered state.*	Summarized Report
expiredEventIdsCanRaise	Verified	*Verifies that requestRaise reverts when the trigger event is not in PENDING_RAISE state. This ensures only valid pending raise events can be used for raise requests.*	

requestRaiseWorksAsExpected	Verified	*Verifies that requestRaise correctly decreases both the global and controller-specific pending raises counts. Furthermore, it the actual contract balance, the asset pool amount and the deposit amount all decrease the same. This ensures accounting is properly maintained when raises are requested.*	

SM-32. Changing numPendingRaises causes state change

Status: Verified	The global numPendingRaises variable is closely connected to the SafetyModules state. We check that when numPendingRaises changes from or to zero the states moves accordingly

Rule Name	Status	Description	Link to rule report
numPendingRaisesChangeCausesStateChange	Verified	*Verifies that changes in the number of pending raises cause appropriate state transitions. When pending raises go from 0 to >0, state transitions to TRIGGERED (unless paused); when going from >0 to 0, state transitions to ACTIVE (unless paused).*	Summarized Report
numPendingRaisesZeroWhenActive	Verified	*Ensures that when the contract is in ACTIVE state, there are no pending raises. This maintains the invariant that active state implies no pending raises.*	

SM-33. Triggering behaves as documented

Status: Verified	Calling trigger does the internal accounting to allow for a raise and moves the SafetyModule to triggered if necessary. It reverts if the trigger id was already used.

Rule Name	Status	Description	Link to rule report

triggerWorksAsExpected	Verified	*Verifies that trigger correctly creates a pending raise event, increases pending raise counts, and requires the caller to be a controller. This ensures trigger operations work as expected and maintain proper accounting.*	*Summarized Report*
triggerWhenTriggeredWithSameIdReverts	Verified	*Verifies that trigger reverts when called with a trigger event ID that has already been triggered. This prevents duplicate triggering of the same event ID.*	
triggerChangesStateProperly	Verified	*Verifies that trigger correctly transitions the state from ACTIVE to TRIGGERED and increases pending raises. This ensures trigger operations cause the expected state changes.*	

SM-34. Only the queued config update can be finalized

Status: Verified	Config updates use a two-step process, the finalization can only work on the updates that were queued before.

Rule Name	Status	Description	Link to rule report
finalizeUpdateConfigsOnlyWithSameParameters	Verified	*Verifies that finalizeUpdateConfigs can only finalize config updates with the same parameters that were queued. This ensures config updates cannot be modified between queuing and finalization.*	*Report*

SM-35. Config updates are reset when necessary

Status: Verified	Config updates are cancelled when the SharedSafetyModule is updated or the triggered state is left.

Rule Name	Status	Description	Link to rule report
configUpdatesAreCancelledWhenNecessary	Verified	*Verifies that config updates are cancelled when the shared safety module is updated or when leaving TRIGGERED state. This ensures pending config updates are properly cleared in appropriate circumstances.*	Report

SM-36. Config updates are restricted

Status: Verified	Config updates obey a waiting period and can not change or remove existing reserve pools, but they can add new ones.

Rule Name	Status	Description	Link to rule report
finalizeUpdateConfigsOnlyWorksInUpdateWindow	Verified	*Verifies that finalizeUpdateConfigs can only be called within the valid update window (between update time and deadline). This ensures config updates can only be finalized during the appropriate time period.*	Summarized Report
finalizeUpdateConfigsCanAddReservePools	Verified	*Verifies that finalizeUpdateConfigs can only be called within the valid update window (between update time and deadline). This ensures config updates can only be finalized during the appropriate time period.*	
finalizeUpdateConfigsCantRemoveOrChangeReservePools	Verified	*Verifies that finalizeUpdateConfigs cannot remove or modify existing reserve pools. This ensures existing pools and their properties remain unchanged during config updates.*	

SM-37. Config updates are checked properly

Status: Verified	Config updates are checked before being queued. They can not change existing reserve pools or include controller addresses already registered for other SafetyModules.

Rule Name	Status	Description	Link to rule report
updateConfigsChecksData	Verified	*Verifies that updateConfigs validates reserve pool and controller configurations before queuing updates. This ensures only safe and valid config updates can be queued.*	Report

SM-38. Updating SharedSafetyModule is restricted

Status: Verified	Finalizing config updates obeys the delay, fails if a new SharedSafetyModule has been proposed and properly updates the SharedSafetyModule.

Rule Name	Status	Description	Link to rule report
finalizeSharedSafetyModuleFailsIfWrongSharedSafetyModule	Verified	*Verifies that finalizeSharedSafetyModule reverts when called with a different shared safety module than was queued. This ensures only the correct queued module can be finalized.*	Summarized Report
finalizeSharedSafetyModuleOnlyWorksInUpdateWindow	Verified	*Verifies that finalizeSharedSafetyModule can only be called within the valid update window and sets the correct module. This ensures shared safety module updates can only be finalized during the appropriate time period.*	

SM-39. Number of controllers is limited

Status: Verified	The safety module imposes an upper limit on the number of registered controllers. Any config update that would exceed this limit reverts. Config

	updates that would temporarily exceed the limit (by first adding a controller and then removing controllers) revert nevertheless.

Rule Name	Status	Description	Link to rule report
limitNumberOfControllers	Verified	*Verifies that the number of controllers never exceeds the allowed limit set by the manager. This ensures controller count remains within configured bounds.*	*Summarized Report*
addControllersAtLimitReverts	Verified	*Verifies that updateConfigs reverts when attempting to add a new controller when the controller limit is already reached. This ensures the controller limit cannot be exceeded even when trying to add a single controller.*	
addAndRemoveControllersReverts	Verified	*Verifies that updateConfigs reverts when attempting to remove one controller and add another when at the controller limit. This highlights a potentially unexpected behavior where updateConfigs reverts although the overall operation would not exceed the controller limit.*	

SM-40. Queued SharedSafetyModule is the right one

Status: Verified	The queueSharedSafetyModule correctly picks up the proposed module.

Rule Name	Status	Description	Link to rule report
queueSharedSafetyModuleUsedProperModule	Verified	*Verifies that queueSharedSafetyModule correctly queues the proposed shared safety module. This ensures the queued module matches what was proposed.*	*Report*

SM-41. Raising scales pending withdrawals

Status: Verified	The queueSharedSafetyModule correctly picks up the proposed module.

Rule Name	Status	Description	Link to rule report
requestRaisesScalesPendingWithdrawals	Verified	*Verifies that requestRaise correctly scales pending withdrawals proportionally when assets are raised. This ensures pending withdrawals are properly adjusted when reserve assets are used for raises.*	Report

SM-42. Pending withdrawals are at most the deposit amount

Status: Verified	The pending withdrawals amount can never exceed the deposit amount. This ensures basic liquidity of the reserve pool.

Rule Name	Status	Description	Link to rule report
depositsGEQpendingWithdrawalsA	Verified	*Ensures that the total deposit amount in a reserve pool is always greater than or equal to pending withdrawals. This invariant applies to all methods except requestRaise, which is handled separately.*	Report
depositsGEQpendingWithdrawalsB	Verified	*Ensures that the total deposit amount in a reserve pool is always greater than or equal to pending withdrawals. This invariant specifically applies to the requestRaise method.*	Report

SM-43. Depositing reserve assets rounds down

Status: Verified	When depositing reserve assets, the amount of receipt tokens payed out is rounded down in favor of the protocol.

Rule Name	Status	Description	Link to rule report
previewDepositReserveAssetsRoundsDown	Verified	*Check that the previewDepositReserveAssets function rounds in favor of the contract. The rule replicates the documented calculation and makes sure the actual result is not larger than that.*	*Report*

RewardsManager

Module General Assumptions

Module Properties

- To cope with rather complex code, many properties are split into multiple rules such that different methods are dealt with in separate prover jobs. Rules that only differ in their trailing letter (A, B, ...) do the same thing but for different methods.

RM-01. Config updates are restricted	
Status: Verified	There is no way to remove stake pools or reward pools or change their assets. For stake pools, their weights sum up to 10000 ("ZOC").

Rule Name	Status	Description	Link to rule report
canNotRemovePoolsOrChangeAssetsA	Verified	*Verifies that finalizeSharedSafetyModule reverts when called with a different shared safety module than was queued. This ensures only the correct queued module can be finalized.*	*Report*
canNotRemovePoolsOrChange	Verified	*Verifies that finalizeSharedSafetyModule can only be called within the valid update window and sets the*	*Report*

AssetsB		*correct module. This ensures shared safety module updates can only be finalized during the appropriate time period.*	
sumOfWeightsIsZOC	Verified	*Verifies that the sum of reward weights across all stake pools equals ZOC (10000). This ensures that reward distribution weights are properly normalized.*	*Report*

RM-02. Depositing assets updates internal accounting

Status: Verified	Calling depositRewardAssets increases the contracts balance (minus the fee) and updates lastDripTime, undrippedRewards and the relevant data in depositorRewards.

Rule Name	Status	Description	Link to rule report
depositRewardAssetsDoesProperAccounting	Verified	*Verifies that depositing reward assets correctly updates all accounting state. The balance increases by the deposit amount minus fees, the last drip time is updated to the current timestamp, undripped rewards increase by the deposit amount minus fees minus any dripped amount, and the depositor's withdrawable rewards, epoch, and log index snapshot are properly updated.*	*Report*

RM-03. Asset pool amount tracks actual balance

Status: Verified	The asset pool amount variable always follows the actual balance in the underlying asset.

Rule Name	Status	Description	Link to rule report

stakePoolsTrackBalanceA	Verified	*Verifies that the asset pool amount correctly tracks the stake balance for a stake pool. Ensures that the amount never exceeds the balance and that changes in amount are bounded by changes in balance. Applies to methods excluding stakeWithoutTransfer, claimRewards, and claimRewardsBySig.*	Report
stakePoolsTrackBalanceB	Verified	*Verifies that the asset pool amount correctly tracks the stake balance for a stake pool. Ensures that the amount never exceeds the balance and that changes in amount are bounded by changes in balance. Applies specifically to claimRewards methods.*	Report
stakePoolsTrackBalanceC	Verified	*Verifies that the asset pool amount correctly tracks the stake balance for a stake pool. Ensures that the amount never exceeds the balance and that changes in amount are bounded by changes in balance. Applies specifically to claimRewardsBySig methods.*	Report
amountIsAtMostBalance	Verified	*Ensures that the asset pool amount never exceeds the stake balance for any stake pool. This invariant must hold at all times during contract execution.*	Report

RM-04. Undripped rewards only increase with balance

Status: Verified	Undripped rewards for any reward pool only increase if the contract's balance within that asset increases. This does not hold for updateConfigs (which moves unclaimed rewards back to undripped rewards) and depositRewardAssetsWithoutTransfer (which explicitly deposits, and thus changes the undripped rewards, without changing the balance as it assumes the transfer has already been done).

Rule Name	Status	Description	Link to rule report
undrippedRewardsOnlyIncreasesWithBalance	Verified	*Ensures that undripped rewards in a reward pool only increase when the reward pool balance increases. This prevents undripped rewards from*	Report

A		*growing without corresponding balance increases. Applies to methods excluding updateConfigs, depositRewardAssetsWithoutTransfer, and claimRewardsBySig.*	
undrippedRewardsOnlyIncreasesWithBalanceB	Verified	*Ensures that undripped rewards in a reward pool only increase when the reward pool balance increases. This prevents undripped rewards from growing without corresponding balance increases. Applies specifically to claimRewardsBySig methods.*	Report

RM-05. Claiming rewards also drips rewards

Status: Verified	Calling any of the several methods to claim rewards first drips rewards before claiming them. This is restricted to the relevant reward pools that are subject to the claiming operation.

Rule Name	Status	Description	Link to rule report
claimRewardsUpdatesLastDripTimeA	Verified	*Verifies that claiming rewards with a single stake pool ID updates the last drip time to the current block timestamp. This ensures that reward distribution timing is properly tracked when rewards are claimed through the single stake pool ID method.*	Summarized Report
claimRewardsUpdatesLastDripTimeB	Verified	*Verifies that claiming rewards with an array of stake pool IDs updates the last drip time to the current block timestamp. This ensures that reward distribution timing is properly tracked when rewards are claimed through the array of stake pool IDs method.*	
claimRewardsUpdatesLastDripTimeC	Verified	*Verifies that claiming rewards with a single stake pool ID and claim rewards pool data (where drip is enabled for the reward pool) updates the last drip*	

		time to the current block timestamp. This ensures that reward distribution timing is properly tracked when rewards are claimed with explicit drip configuration for the reward pool.	
claimRewardsUpdatesLastDripTimeD	Verified	*Verifies that claiming rewards with an array of stake pool IDs and claim rewards pool data (where drip is enabled for the reward pool) updates the last drip time to the current block timestamp. This ensures that reward distribution timing is properly tracked when rewards are claimed with multiple stake pools and explicit drip configuration for the reward pool.*	
claimRewardsUpdatesBySigLastDripTimeA	Verified	*Verifies that claiming rewards by signature with an array of stake pool IDs updates the last drip time to the current block timestamp. This ensures that reward distribution timing is properly tracked when rewards are claimed through the signature-based method with multiple stake pools.*	
claimRewardsUpdatesBySigLastDripTimeB	Verified	*Verifies that claiming rewards by signature with an array of stake pool IDs and claim rewards pool data (where drip is enabled for the reward pool) updates the last drip time to the current block timestamp. This ensures that reward distribution timing is properly tracked when rewards are claimed through the signature-based method with multiple stake pools and explicit drip configuration.*	

RM-06. Rewards drip only when the manager is active

Status: Verified	Rewards are only dripped when the state of the RewardsManager is ACTIVE.

Rule Name	Status	Description	Link to rule report

canDripOnlyWhenActiveA	Verified	*Ensures that rewards can only drip when the rewards manager is in ACTIVE state. Verifies that if dripped rewards increase, the state must be ACTIVE either before or after the method call. Applies to all methods except claimRewardsBySig.*	Report
canDripOnlyWhenActiveB	Verified	*Ensures that rewards can only drip when the rewards manager is in ACTIVE state. Verifies that if dripped rewards increase, the state must be ACTIVE either before or after the method call. Applies specifically to claimRewardsBySig methods.*	Report

RM-07. Claiming rewards transfers the correct amounts

Status: Verified after fix	The amount transferred to the receiver when calling claimRewards matches the preview result for claimable rewards.

Rule Name	Status	Description	Link to rule report
claimRewardsWorksAsExpected	Verified after fix	*Verifies that claiming rewards correctly transfers the expected amount from the reward pool balance. Ensures that the decrease in reward pool balance equals the sum of the claimed amount and claim fee amount as previewed.*	Report

RM-08. Claiming rewards claims the full amount

Status: Verified	Calling claimRewards claims all rewards claimable for this user and the claimable rewards are zero afterwards.

Rule Name	Status	Description	Link to rule report

claimRewardsClaimsEverything	Verified	*Ensures that after claiming rewards, all claimable rewards amounts are zero. Verifies that claimRewards claims all available rewards for the user.*	Report

RM-09. Receipt token supply matches stake pool amount

Status: Verified	The amount in a stake pool should always be identical to the total supply in its stake receipt token.

Rule Name	Status	Description	Link to rule report
stakePoolAmountTracksSupply	Verified	*Ensures that the stake pool amount always equals the total supply of stake receipt tokens. This invariant must hold at all times, maintaining consistency between the stake pool state and token supply.*	Report

RM-10. Index snapshots can only increase

Status: Verified	The index snapshots of any claimable rewards always increases.

Rule Name	Status	Description	Link to rule report
claimableRewardsIndexSnapshotIsIncreasingA	Verified	*Ensures that the claimable rewards index snapshot is non-decreasing over time. Verifies that the index snapshot can only increase or remain constant, never decrease. Applies to all methods except claimRewardsBySig.*	Report
claimableRewardsIndexSnapsh	Verified	*Ensures that the claimable rewards index snapshot is non-decreasing over time. Verifies that the index*	Report

otIsIncreasingB		*snapshot can only increase or remain constant, never decrease. Applies specifically to claimRewardsBySig methods.*	

RM-11. Each pool can only be claimed once in one call

Status: Verified	Claiming the same reward pool twice in one call to claimRewards is rejected. It relies on _checkValidClaimRewardsPoolData being called properly by claimRewards.

Rule Name	Status	Description	Link to rule report
cantClaimSamePoolTwice	Verified	*Ensures that claim rewards pool data validation rejects arrays containing duplicate reward pool IDs. This prevents invalid claim operations where the same reward pool is specified multiple times in a single claim transaction, which could lead to accounting errors or double-claiming.*	Report

RM-12. Index snapshots for user rewards are at most those of claimable rewards

Status: Verified	The index snapshots of any claimable rewards always increases.

Rule Name	Status	Description	Link to rule report
userRewardsSnapshotLEQClaimableRewardsSnapshotA	Verified	*Ensures that a user's rewards index snapshot never exceeds the claimable rewards index snapshot for any stake pool and reward pool combination. This invariant guarantees that users cannot have a snapshot index that is ahead of the global claimable rewards snapshot, preventing them from claiming rewards that have not yet been made available. The invariant is split into three variants to cover all*	Report
userRewardsSnapshotLEQClaimableRewards	Verified		Report

SnapshotB		*methods: general methods, claimRewards methods, and claimRewardsBySig methods.*	
userRewardsSnapshotLEQClaimableRewardsSnapshotC	Verified		Report

RM-13. Stake pool get rewards based on weight

Status: Verified	The rewards are distributed among the different stake pools according to their relative weight.

Rule Name	Status	Description	Link to rule report
stakePoolsGetWeightedRewardsA	Verified	*Ensures that stake pools receive at most their weighted share of cumulative dripped rewards. Verifies that cumulative claimable rewards for a stake pool do not exceed the pool's weight times cumulative dripped rewards divided by 10000. Applies to methods excluding claimRewards, claimRewardsBySig, and initialize.*	Report
stakePoolsGetWeightedRewardsB	Verified	*Ensures that stake pools receive at most their weighted share of cumulative dripped rewards. Verifies that cumulative claimable rewards for a stake pool do not exceed the pool's weight times cumulative dripped rewards divided by 10000. Applies specifically to the claimRewardsBySig method with uint16[] parameter.*	Report
stakePoolsGetWeightedRewardsC	Verified	*Ensures that stake pools receive at most their weighted share of cumulative dripped rewards. Verifies that cumulative claimable rewards for a stake pool do not exceed the pool's weight times cumulative dripped rewards divided by 10000. Applies specifically to the claimRewardsBySig method with ClaimRewardsPoolData[] parameter.*	Report

stakePoolsGetWeightedRewardsD	Verified	*Ensures that stake pools receive at most their weighted share of cumulative dripped rewards. Cerifies that cumulative claimable rewards for a stake pool do not exceed the pool's weight times cumulative dripped rewards divided by 10000. Applies specifically to claimRewards methods.*	Report
stakePoolsGetWeightedRewardsE	Verified	*Ensures that stake pools receive at most their weighted share of cumulative dripped rewards. Verifies that cumulative claimable rewards for a stake pool do not exceed the pool's weight times cumulative dripped rewards divided by 10000. Applies specifically to claimRewards methods.*	Report

Euler integration

EI-01. Liquidator is only called in the right state

Status: Verified	If the Euler eVault calls the liquidator and bad debt socialization is enabled, the call must be reentrant as per the liquidators _entered flag. If the caller is not the Euler eVault, reentrancy is not allowed.

Rule Name	Status	Description	Link to rule report
eVaultCannotCallLiquidateIfDebtsAreSocialized	Verified	*Ensures that when the eVault calls liquidate, if debts are socialized (CFG_DONT_SOCIALIZE_DEBT flag is set), the contract must have been entered before (reentrancy protection). For non-eVault callers, the contract should not have been entered before the call.*	Report

EI-02. EVault liquidation do not change the state

Status: Verified	If the Euler eVault calls liquidate, no state changes are allowed. This effectively disallows any business logic, the code should just return early.

Rule Name	Status	Description	Link to rule report
eVaultLiquidateCausesNoStateChange	Verified	*Verifies that when the eVault calls liquidate, it does not modify the contract's state. Specifically, the _entered flag and triggerEventIdNonce must remain unchanged.*	Report

EI-03. Raise only on bad debt

Status: Verified	The liquidator should only raise on the SafetyModule if there is debt without any collateral.

Rule Name	Status	Description	Link to rule report
onlyRaiseAfterFullLiquidation	Verified	*Ensures that a raise event (indicated by a change in triggerEventIdNonce) only occurs after a full liquidation. The violator must have had debt but no collateral before the operation that triggers the raise.*	Report

EI-04. Always take funds from the reserve pool first

Status: Verified	The EulerTrancheRaiseStrategy first uses all funds from its fee pool and only then uses the lender pool to satisfy any further demands.

Rule Name	Status	Description	Link to rule report

tranchingWorksCorrectly	Verified	*Verifies that calculateRaise correctly implements the tranching logic, prioritizing the fee pool first and then the lender pool. The rule ensures proper amount calculations based on available pool balances, with the fee pool being exhausted before drawing from the lender pool.*	Report

Module controllers

MC-01. Only the owner can raise

Status: Verified	Only the owner of an OwnableController and call the triggerAndRequestRaise function.

Rule Name	Status	Description	Link to rule report
onlyOwnerCanTrigger	Verified	*Verifies that only the contract owner can trigger a raise request. This rule ensures that the triggerAndRequestRaise function enforces access control by checking that the message sender is the owner.*	Report

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.